

AI-BASED CODING ASSESSMENT PLATFORM

Software Requirements Specification (SRS)

Document Type	Software Requirements Specification
Project Domain	Artificial Intelligence / Education / Full-Stack Development
Primary Application	AI-Assisted Coding Assessment and Personalized Learning
Development Stack	MERN (MongoDB, Express.js, React, Node.js)
AI Integration	Krutrim Cloud / Qwen (proposed)
Document Version	1.0
Status	Requirements Baseline

This document defines the functional, non-functional, technical, AI, assessment, security, data, evaluation, and deployment requirements for the AI-Based Coding Assessment Platform. The structure follows the style and level of detail of the provided SRS reference document.

1. Introduction

Programming education and assessment often rely on manual checking, fixed-difficulty coding questions, and basic test-case results. These approaches can evaluate whether a program works, but they may provide limited information about a student's coding quality, problem-solving approach, weak areas, or next learning steps.

The proposed AI-Based Coding Assessment Platform focuses on automated programming assessment combined with AI-assisted code analysis and personalized learning support. The platform is intended to provide a practical environment for coding tests, automated execution, AI-generated feedback, adaptive assessments, and skill analysis.

The system is designed for students, teachers, and educational institutions and aims to make coding assessment more efficient while helping students understand and improve their programming skills.

2. Purpose

The purpose of the system is to assess a student's programming ability through coding problems, automated test-case evaluation, and AI-assisted analysis.

The system shall accept programming solutions, execute them in a controlled assessment environment, compare the results with defined test cases, and return assessment results. Where enabled, the AI component shall analyze the submitted code and generate understandable feedback, hints, explanations, and improvement suggestions.

The platform shall also support performance tracking and personalized recommendations so that assessment can contribute to continuous learning rather than only producing a score.

3. System Scope

3.1 In Scope

- User registration and login
- Student profile and skill tracking
- Teacher/admin account and assessment management
- Coding problem creation and management
- Online code editor
- Programming language selection
- Code submission and automated test-case evaluation
- Execution result and score display
- AI-based code review and explanation
- AI-generated hints and improvement suggestions
- Adaptive question difficulty
- Skill analysis and performance reports
- Personalized learning recommendations
- Assessment history
- Teacher dashboard and class-level analytics
- REST API for frontend-backend communication
- MongoDB-based data storage
- Role-based access control
- Secure handling of AI API keys and credentials

3.2 Out of Scope

- Guaranteed detection of every form of plagiarism or cheating

- Unrestricted execution of arbitrary system commands
- Direct access to a student's private files without permission
- Training a foundation AI model from scratch in the initial release
- Guaranteed correctness of AI-generated explanations
- Automatic grading based only on an AI response without test-case evidence
- Production-scale multi-tenant deployment beyond the project requirements

4. System Overview

The proposed architecture separates presentation, application services, code evaluation, database access, AI integration, and analytics so that each component can be developed and tested independently.

Layer / Component	Connected Component	Primary Responsibility
Student / Teacher	Web Interface (React)	Authentication, dashboard, coding editor, reports
React Frontend	Node.js + Express.js API	Assessment, users, submissions, AI requests
Express Backend	Code Execution / Evaluation Layer	Run code, test cases, collect results
Express Backend	MongoDB	Users, problems, assessments, submissions, results
Express Backend	Krutrim Cloud / Qwen	AI review, hints, explanations, recommendations
Backend	Assessment Analytics	Scores, skills, progress and performance history

5. Functional Requirements

FR-001 — User Registration and Login — The system shall provide secure registration and login for authorized users.

FR-002 — Role Management — The system shall support appropriate roles such as Student and Teacher/Admin.

FR-003 — Problem Management — Authorized users shall be able to create, update, categorize, and manage coding problems.

FR-004 — Assessment Creation — Teachers/Admins shall be able to create assessments using selected coding problems, time limits, difficulty levels, and test cases.

FR-005 — Code Editor — The system shall provide an online coding editor for writing and editing solutions.

FR-006 — Language Selection — The system shall allow a problem to specify supported programming languages and allow the student to select an available language.

FR-007 — Code Submission — The system shall accept a student's code submission and associate it with the relevant user, problem, and assessment.

FR-008 — Automated Evaluation — The system shall execute submitted code in a controlled evaluation environment and compare outputs against configured test cases.

FR-009 — Result Calculation — The system shall calculate and display test-case results, score, execution information, and assessment status.

FR-010 — AI Code Review — Where enabled, the system shall send appropriate code and assessment context to the configured AI service and return code-quality feedback.

FR-011 — AI Hints — The system should provide progressive hints that guide the student without directly revealing the complete solution.

FR-012 — AI Explanation — The system should explain important parts of the submitted code and identify possible improvements in understandable language.

FR-013 — Adaptive Assessment — The system should adjust subsequent question difficulty using defined performance rules and recorded assessment results.

FR-014 — Skill Analysis — The system shall maintain skill-related assessment data and identify areas of strength and weakness.

FR-015 — Recommendations — The system should provide personalized learning recommendations based on assessment performance and identified weak areas.

FR-016 — History and Reports — The system shall provide students with assessment history and performance reports.

FR-017 — Teacher Analytics — Authorized teachers shall be able to view relevant student and class-level assessment analytics.

FR-018 — Error Handling — The system shall return meaningful errors for invalid requests, failed code execution, unavailable AI services, and internal failures.

6. Coding Assessment Requirements

CODE-001 — Test Case Definition — Each coding problem shall support visible and/or hidden test cases as appropriate.

CODE-002 — Input Handling — The evaluation service shall provide problem-specific input to submitted programs according to the configured test case.

CODE-003 — Output Validation — Program output shall be compared with the expected result using the configured evaluation rule.

CODE-004 — Execution Limits — The evaluation environment shall apply appropriate time and resource limits.

CODE-005 — Compilation Errors — Compilation or syntax errors shall be captured and returned as understandable assessment information.

CODE-006 — Runtime Errors — Runtime failures and abnormal termination shall be recorded without crashing the platform.

CODE-007 — Submission Record — Each submission shall record user, problem, language, timestamp, result, score, and relevant execution metadata.

CODE-008 — Correctness Priority — Final correctness shall be determined primarily through controlled execution and test-case results; AI feedback shall supplement rather than replace objective execution results.

7. AI Requirements

AI-001 — AI Service Integration — The backend shall support integration with a configured AI service such as Krutrim Cloud/Qwen through a secure server-side API integration.

AI-002 — Code Review — The AI service shall analyze submitted code for readability, structure, possible inefficiencies, and improvement opportunities where supported.

AI-003 — Feedback Generation — The system should generate concise, understandable feedback linked to the student's submission and assessment result.

AI-004 — Hint Generation — The system should provide hints at progressive levels, avoiding direct disclosure of the complete solution when the assessment policy requires it.

AI-005 — Code Explanation — The system should explain the logic of a student's solution and highlight important programming concepts.

AI-006 — Personalized Recommendation — The system should generate learning recommendations from stored assessment and skill data.

AI-007 — Prompt Context — The backend shall provide only the necessary problem statement, code, results, and context to the AI service.

AI-008 — AI Output Handling — AI-generated content shall be treated as assistance and shall not be considered an unquestionable source of truth.

AI-009 — Failure Handling — If the AI service is unavailable, the core coding assessment shall continue where possible without AI feedback.

AI-010 — Usage Control — The system should support configurable limits for AI requests to control cost, latency, and misuse.

8. User, Data and Database Requirements

DATA-001 — User Data — The system shall store required user profile, role, authentication-related, and academic/assessment information.

DATA-002 — Problem Data — The system shall store problem statements, difficulty, topics, supported languages, constraints, and test cases.

DATA-003 — Submission Data — The system shall store relevant code submission and evaluation information according to the retention policy.

DATA-004 — Assessment Data — The system shall store assessment configuration, attempts, scores, and completion status.

DATA-005 — AI Feedback Data — Where enabled, relevant AI feedback and metadata may be stored for later review and learning support.

DATA-006 — Skill Data — The system shall maintain topic/skill performance indicators derived from completed assessments.

DATA-007 — Data Quality — The application shall validate required fields, detect malformed requests, and prevent inconsistent records.

DATA-008 — Database — MongoDB shall be used as the primary application database for the proposed MERN implementation.

9. Assessment and Skill Evaluation Requirements

EVAL-001 — Score Calculation — The system shall calculate scores using configured test-case results and assessment rules.

EVAL-002 — Performance Metrics — The system shall track metrics such as accuracy, completion rate, execution time where available, attempts, and topic-wise performance.

EVAL-003 — Difficulty Adaptation — The adaptive module should use previous performance to select an appropriate difficulty level or question category.

EVAL-004 — Skill Profile — The system should maintain a skill profile covering relevant programming topics such as arrays, strings, recursion, data structures, algorithms, debugging, and problem solving.

EVAL-005 — Progress Tracking — The system shall show changes in performance over completed assessments.

EVAL-006 — AI Feedback Quality — AI feedback should be evaluated for relevance, clarity, consistency, and usefulness during project testing.

EVAL-007 — Comparative Evaluation — The project evaluation should compare the proposed workflow with a basic automated coding assessment workflow where practical.

10. Web Application and API Requirements

WEB-001 — Student Interface — The application shall provide interfaces for login, dashboard, assessment selection, coding, submission, results, history, and recommendations.

WEB-002 — Teacher Interface — The application should provide interfaces for problem/assessment management and relevant analytics.

WEB-003 — Responsive Interface — The interface should remain usable on common desktop and mobile screen sizes.

WEB-004 — API Integration — The React frontend shall communicate with the Node.js/Express backend through defined APIs.

API-001 — Input Validation — All API inputs shall be validated before processing.

API-002 — Structured Responses — The API shall return consistent machine-readable response structures.

API-003 — Authentication — Protected API routes shall verify the authenticated user's access rights.

API-004 — Error Codes — The API shall provide meaningful HTTP status codes and error messages.

API-005 — AI Endpoint Protection — AI-related endpoints shall not expose provider API keys to the browser.

11. Non-Functional Requirements

NFR-001 — Performance — Normal coding submissions and application requests should provide interactive response times appropriate for the available infrastructure; measured latency shall be reported as a project result.

NFR-002 — Reliability — Invalid submissions, code failures, database errors, and temporary AI-service failures shall be handled without terminating the application.

NFR-003 — Maintainability — Frontend, backend, database access, assessment, code execution, and AI integration components shall be modular.

NFR-004 — Scalability — The architecture should permit additional problems, languages, users, AI features, and analytics without redesigning the complete system.

NFR-005 — Portability — The application should support standard Windows/Linux development environments and suitable cloud deployment.

NFR-006 — Usability — Assessment screens, results, and AI feedback shall be presented in a clear and understandable manner.

NFR-007 — Extensibility — The system should allow additional AI models or providers to be integrated through a controlled backend layer.

12. Security and Privacy Requirements

SEC-001 — Authentication — The system shall authenticate users before allowing access to protected functions.

SEC-002 — Authorization — Users shall only access resources permitted by their role.

SEC-003 — Input Sanitization — User-provided input shall be validated and safely handled.

SEC-004 — Code Execution Isolation — Submitted code shall be executed in a controlled environment with appropriate restrictions; the application shall not grant unnecessary host-system privileges.

SEC-005 — Secret Management — AI API keys, database credentials, JWT secrets, and other sensitive configuration shall not be hard-coded or committed to GitHub.

SEC-006 — Rate Limiting — Production APIs should apply suitable rate limits to reduce abuse.

SEC-007 — Logging — Important application and security events shall be logged without unnecessarily storing sensitive user information.

PRIV-001 — Data Minimization — Only information required for assessment, analytics, AI assistance, and explicitly enabled reporting shall be stored.

PRIV-002 — Retention — Stored submissions and AI feedback should follow a configurable retention policy.

PRIV-003 — Access Control — Assessment records and personal information shall be accessible only to authorized users and components.

13. Hardware Requirements

Component	Minimum	Recommended
CPU	4 cores	6–8 cores
RAM	8 GB	16 GB+
Storage	20 GB free	50 GB+
GPU	Not required for the MERN application; optional for local AI experimentation	Optional
Network	Internet connection	Stable broadband

14. Software and Technology Requirements

Category	Recommended Technology
Frontend	React, HTML, CSS, JavaScript
Backend	Node.js, Express.js
Database	MongoDB / MongoDB Atlas
AI	Krutrim Cloud / Qwen or another configured AI provider
Code Editor	Monaco Editor or equivalent web editor
Authentication	JWT/session-based authentication as implemented
API	REST API
Version Control	Git and GitHub
Development	VS Code, npm

15. System Constraints

- Assessment quality depends on the quality and coverage of coding problems and test cases.
- Code execution must be isolated and restricted; unsafe execution can create security risks.
- AI-generated feedback can be incorrect or incomplete and must not replace objective test-case evaluation.
- AI response quality depends on the selected model, prompt design, available context, latency, and service availability.
- External AI services may impose token limits, rate limits, costs, or availability constraints.
- Adaptive assessment quality depends on meaningful skill labels and sufficient assessment history.
- Project-scale infrastructure may not support the same concurrency as commercial coding platforms.
- Some advanced programming languages or execution environments may require additional runtime configuration.

16. Future Extensions

- AI-generated coding question creation with teacher approval
- More advanced adaptive assessment using learning analytics
- Human-AI collaboration assessment and AI fluency scoring
- Repository/GitHub-based real-world codebase assessments
- Personalized developer skill passport
- AI-powered mock coding interviewer
- Advanced plagiarism and integrity analysis
- More programming languages and isolated execution workers
- Learning analytics dashboards and predictive skill growth
- Fine-tuned or domain-specific AI models

17. Acceptance Criteria

- A user can register/login and access authorized platform features.
- A coding problem can be created and displayed correctly.
- A student can write and submit code through the web interface.
- Submitted code is evaluated using configured test cases.
- Compilation/runtime failures are handled and displayed safely.
- A score and test-case result are returned for a valid submission.
- AI feedback can be requested without exposing the AI provider API key to the client.
- AI feedback, hints, or explanations are displayed in understandable form when the AI service is available.
- Assessment history and relevant performance information are stored and retrievable.
- Skill-related performance can be summarized for the student.
- Adaptive question selection works according to the implemented rules.
- Teacher/admin users can manage appropriate problems or assessments.
- Protected APIs enforce authentication and authorization.
- Sensitive configuration is kept outside source control.
- The application handles database, evaluation, and AI-service errors without unexpected termination.

18. Requirements Traceability Matrix

Requirement Area	Implementation Component	Validation
User Authentication	Frontend + Auth/API layer	Login and authorization tests
Problem Management	Teacher/Admin module + API	CRUD and validation tests
Code Editor	React editor	Editor and submission UI tests
Code Evaluation	Execution/Evaluation layer	Test-case execution tests
Scoring	Assessment service	Score calculation tests
AI Review	AI service integration	Feedback quality and API tests
Adaptive Assessment	Assessment/analytics module	Difficulty-selection tests
Skill Analysis	Analytics module + MongoDB	Topic-wise performance validation
Database	MongoDB	CRUD and data-integrity tests
Security	Auth/API/execution layer	Security and access-control tests
Performance	Backend/evaluation services	Latency and resource benchmarks
Reporting	Report/history module	Generated report validation

19. Research References

- HackerRank — AI-assisted assessment and coding assessment capabilities. Used as a reference point for understanding existing AI-enabled coding assessment features.
- CodeSignal — AI-assisted assessment and evaluation capabilities. Used as a reference point for existing AI-based assessment workflows.
- Codility — Online coding assessment and AI-assisted coding assessment capabilities. Used as a reference point for current assessment practices.
- Edwards, S. H., and Pérez-Quiñones, M. A., “Web-CAT: Automatically Grading Programming Assignments.” Used as a research reference for automated programming assessment and feedback concepts.
- Krutrim Cloud documentation — AI Studio, model inference, model API keys, and AI model integration. Used as a technical reference for proposed AI integration.